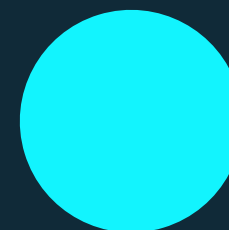
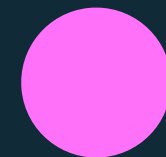




**Não deixe de
registrar a sua
participação!**

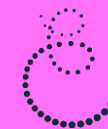


PRÁTICA DE VIBE CODING EM PROJETOS REAIS



1

Como usar IA no fluxo dev
sem virar passageiro do
próprio código



ecossistema de
aprendizagem



Objetivos da aula

Ao final da aula, você deve conseguir:



- Escolher ferramentas de IA para tarefas dev
- Escrever prompts técnicos com mais clareza



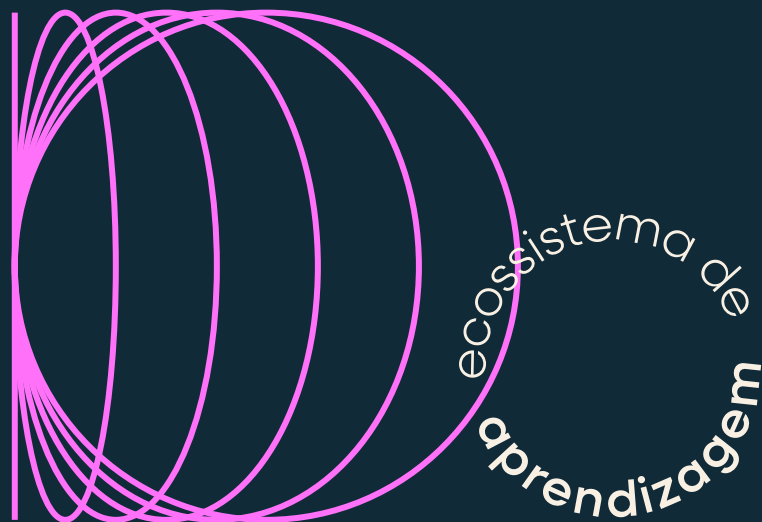
- Gerar funções, apis simples e automações internas
- Validar saídas da IA com testes



- Refatorar código gerado por IA
- Documentar soluções para trabalho em equipe



natura



Prática de Vibe Coding em Projetos Reais: IA aplicada ao fluxo dev

1

O que vamos construir hoje?



Mini fluxo prático

Simulação de tarefas reais do dia a dia, focando na agilidade e precisão técnica que a IA proporciona.

Rule em Markdown

Função de Produto

API Mockada (Node.js)

Testes e Documentação

Onde isso aparece no dia a dia?



Situações comuns em um time dev

- Criar componentes de interface
- Consumir apis
- Ajustar regras de negócio
- Montar scripts internos
- Entender código legado
- Criar testes
- Documentar endpoints
- Revisar pull requests

Foco da aula:

- produtos
- catálogo
- estoque
- campanhas
- automações internas





Ferramentas de Vibe Code: Onde a IA entra?



ChatGPT: Planejar, Revisar e Documentar



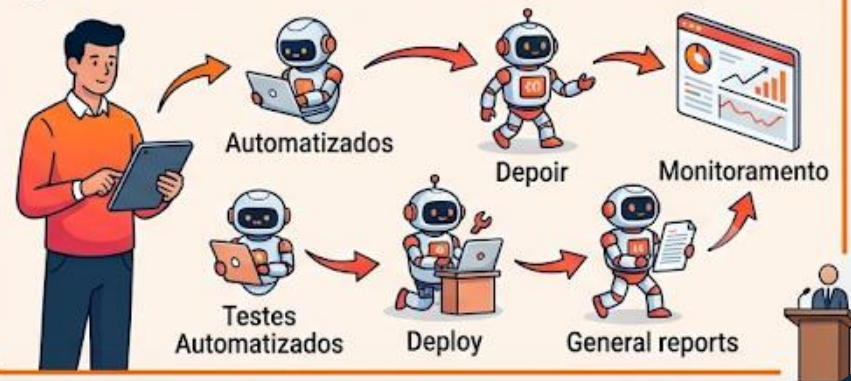
Copilot: Completar Código na IDE



Cursor/Windsurf: Editar Arquivos com Contexto



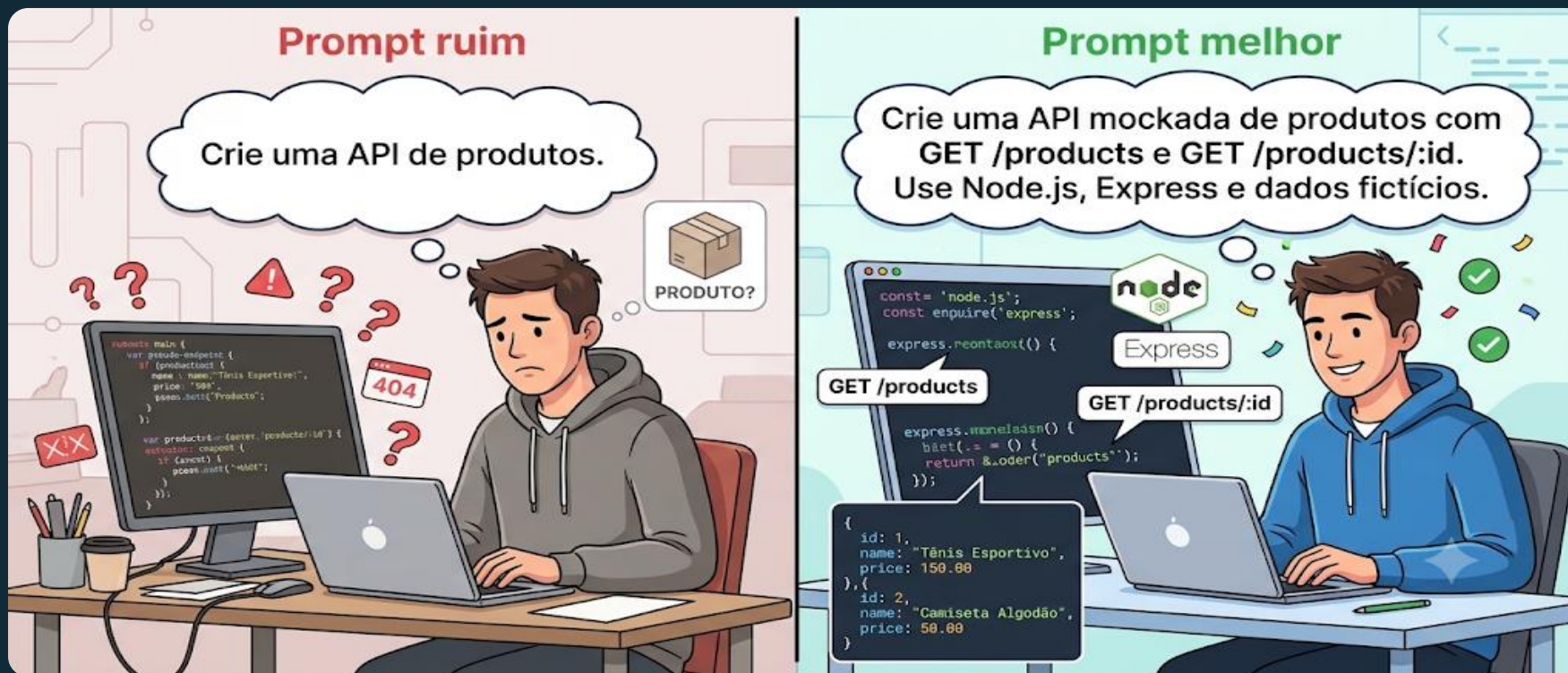
Agentes: Executar Tarefas Maiores com Supervisão



ecossistema de
aprendizagem

Regra prática

IA não trabalha bem no escuro





O que é uma rule em .md?

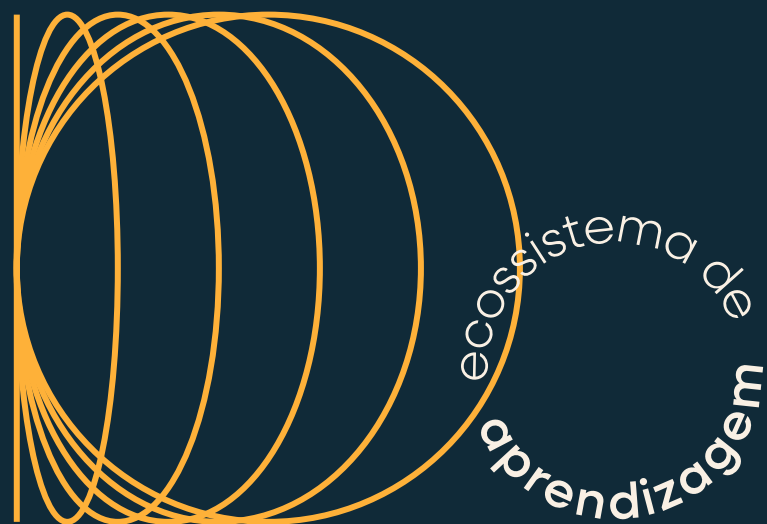
Uma rule é um guia para a IA

Ela define:

- contexto do projeto
- stack usada
- padrões esperados
- regras de negócio
- limites de segurança
- formato da resposta



ecossistema de
aprendizagem



Prática 1:

Criar a rule

2



Criando a Rule

Arquivo:

natura-product-dev.rules.md

Objetivo

Criar regras para desenvolver soluções ligadas a:

- produtos
- catálogo
- estoque
- Campanhas
- exibição para cliente

ecossistema de
aprendizagem



Modelo da Rule

Rule: desenvolvimento com IA para produtos

Contexto

Estou criando soluções relacionadas a produtos, catálogo, estoque e campanhas.

Stack

- React
- TypeScript
- Node.js
- Express

Dados de produto

- id
- name
- imageUrl
- category
- price
- stockStatus

Regras

- não usar dados reais
- não expor tokens ou URLs internas
- validar entradas
- tratar imagem ausente
- formatar preço em BRL
- bloquear ação se produto estiver indisponível

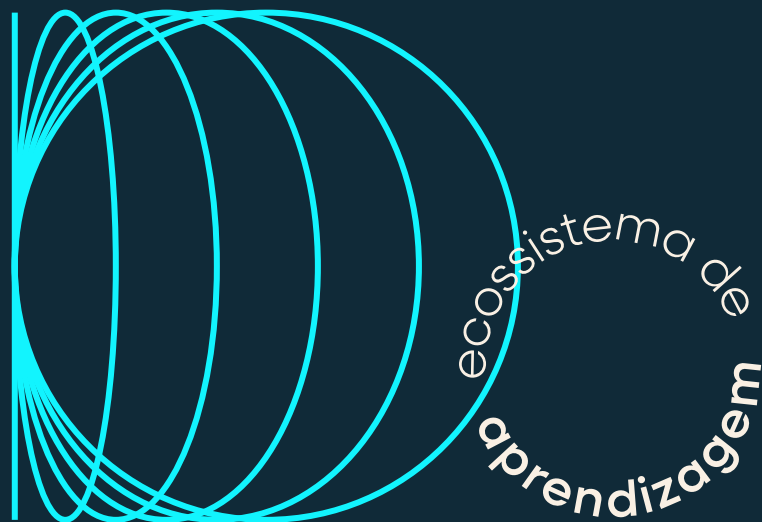
Saída esperada

- código simples
- TypeScript correto
- explicação curta
- sugestões de teste

ecossistema de
aprendizagem



natura



Prática 2:

gerar uma função

3

Gerando uma função



Prompt

Use a rule natura-product-dev.rules.md.

Crie uma função TypeScript chamada normalizeProduct.

Entrada:

```
{
  id: number,
  name: string,
  imageUrl?: string,
  category: string,
  price: number,
  stockStatus: "available" | "unavailable"
}
```

Regras:

- aplicar fallback para imageUrl vazio
- formatar price em BRL
- validar preço negativo
- retornar objeto pronto para exibição
- sugerir casos de teste

```
    }, proxyHandler); data.author = "João";
    ('Project: ${this.name}, Version: ${t
ails(); async function fetchData(url)
se.ok) throw new Error("Network respo
on(); return data; } catch (error) {
numberGenerator() { let num = 1; whil
```




```
Handler); data.author =  
  ${this.name}, Version: $  
ync function fetchData(ur  
ow new Error("Network res  
rn data; } catch (error  
ator() { let num = 1;  
erator(); document.getEle  
ument.getElementById("num  
getElementById("calculate"  
"calculate", payload: 10  
this.fibonacci(n) +  
se, exp); } stati
```

ecossistema de
aprendizagem

Saída esperada

O código deve ter:

- Interface ou type para produto
- Validação de preço
- Fallback de imagem
- Formatação em moeda brasileira
- Retorno simples para interface
- Sugestões de teste



CHECKLIST

☒	
☒	
☒	
☒	
☐	
☐	

ecossistema de
aprendizagem

Checklist de revisão da função

Antes de aceitar o código, observe:

- ☐ O typescript está correto?
- ☐ O preço negativo gera erro?
- ☐ Imagem vazia tem fallback?
- ☐ O preço está em brl?
- ☐ O retorno está fácil de usar?
- ☐ A ia inventou alguma dependência?
- ☐ Dá para explicar o código?

Prompt de ajuste



Ajustando a resposta da IA
Revise a função normalizeProduct.

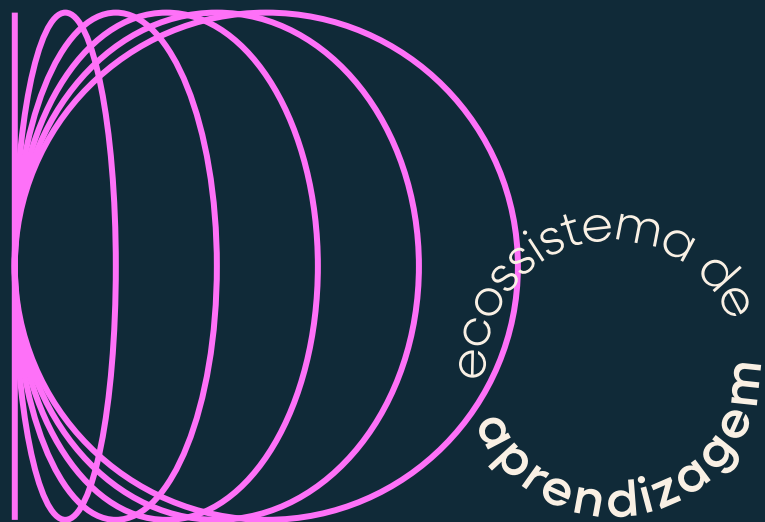
Corrija apenas estes pontos:

- não usar bibliotecas externas
- garantir fallback para imageUrl vazio
- lançar erro quando price for negativo
- manter TypeScript simples

Refinando a Entrega

O Vibe Coding é iterativo. Se a primeira versão não for perfeita, dê feedback específico:

```
"Corrija apenas estes pontos: não usar libs externas,  
lançar erro se price for negativo."
```



Prática 3:

gerar API mockada

Gerando API mockada



Prompt:

Use a rule natura-product-dev.rules.md.

Crie uma API mockada em Node.js com Express.



Rotas:

- GET /products
- GET /products/:id



Regras:

- usar array em memória
- retornar 404 se produto não existir
- não usar banco de dados
- não expor dados sensíveis
- sugerir testes manuais para as rotas



Checklist da API

Verificar se a API tem:

- ☐ Express configurado
- ☐ Array de produtos fictícios
- ☐ Rota para listar produtos
- ☐ Rota para buscar por ID
- ☐ Tratamento de produto inexistente
- ☐ Status HTTP correto
- ☐ Dados seguros e fictícios

ecossistema de
aprendizagem

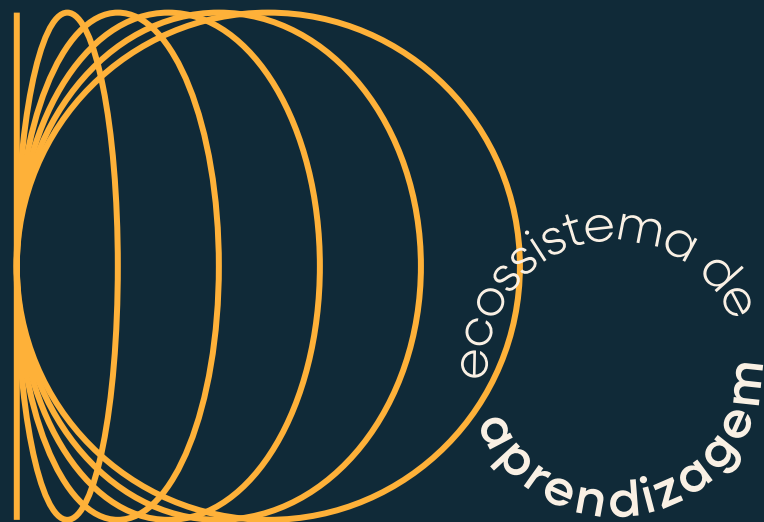
Teste manual da API



Cenário de Teste	Rota (Endpoint)	Resultado Esperado
Listar tudo	GET /products	Array com todos os produtos (Status 200)
Produto existente	GET /products/1	Objeto do produto específico (Status 200)
Produto inexistente	GET /products/999	Mensagem de erro amigável (Status 404)



natura



Prática 4:

pedir testes para a IA

5

Gerando testes com IA

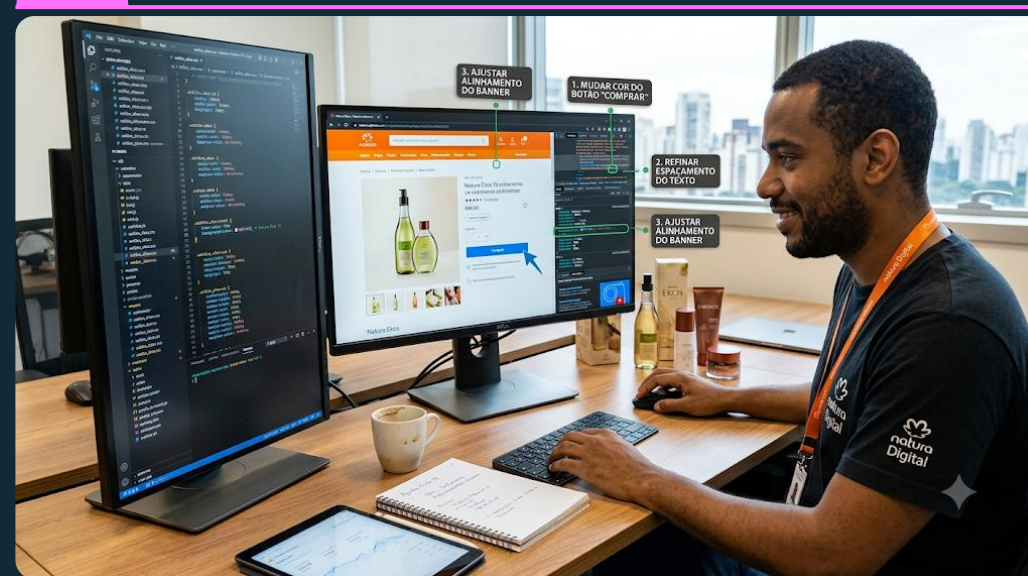


Prompt

Com base nessa API, gere uma lista de testes manuais e testes automatizados.

Considere:

- listagem de produtos
- busca por produto existente
- busca por produto inexistente
- formato da resposta
- status HTTP esperado





Validação de testes

Teste bom verifica comportamento



Teste Útil

"GET /products/999 deve retornar 404."

Verifica comportamento real.

Teste Fraco

"A API deve funcionar."

Genérico, não valida nada.

Ajuste iterativo: Quando a IA erra

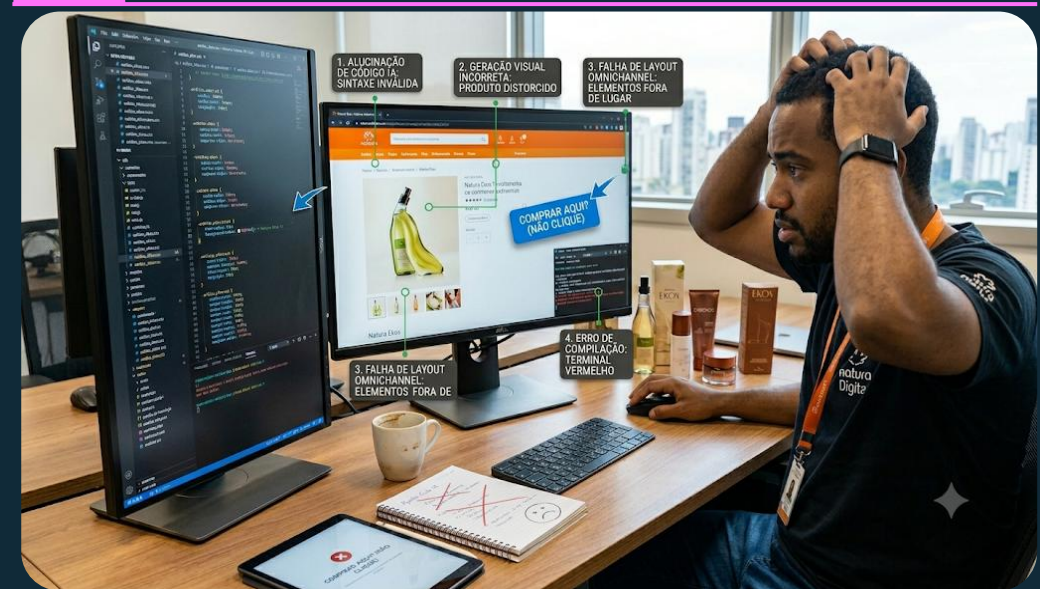


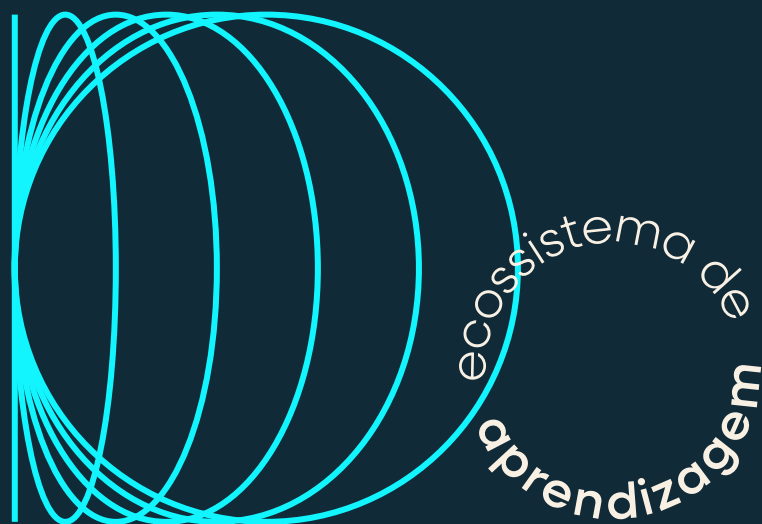
Use prompts específicos:

Os testes estão muito genéricos.

Reescreva com:

- entrada
- ação
- resultado esperado
- status HTTP





Prática 5:

refatorar

6

Refatorar

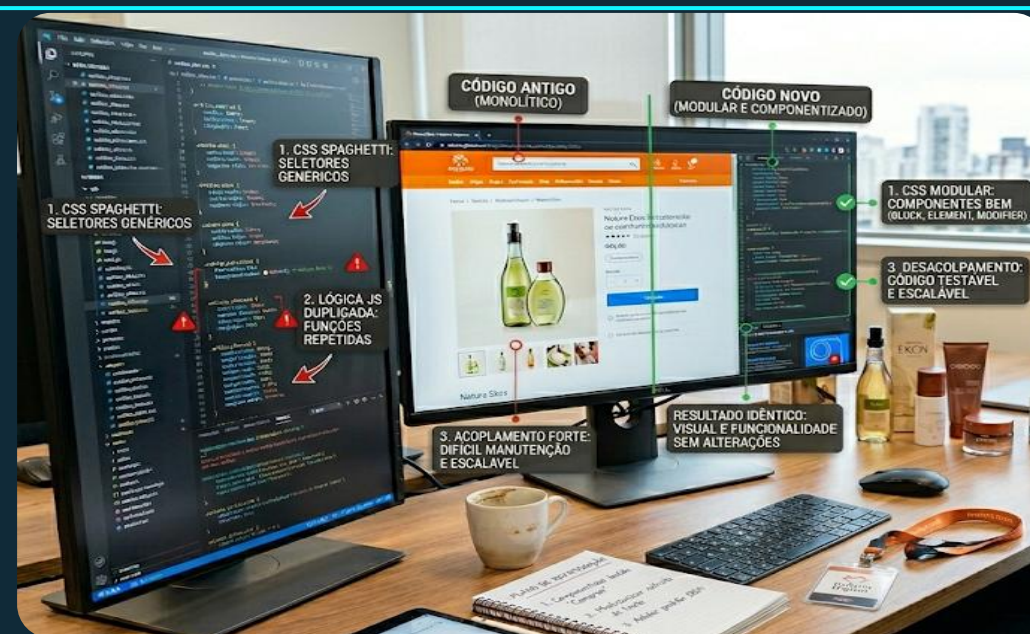


Prompt:

Refatore a API gerada.

Objetivo:

- separar dados mockados em outro arquivo
- criar função findProductById
- manter o mesmo comportamento
- não adicionar bibliotecas
- explicar as mudanças





CHECKLIST

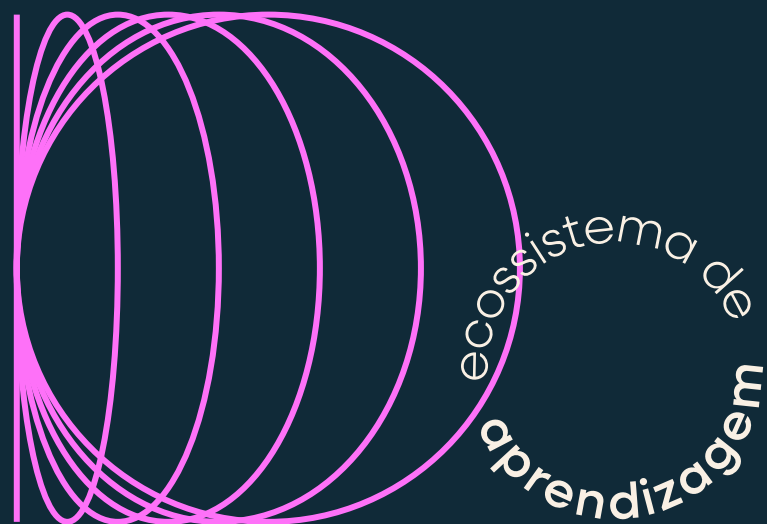
☒	
☒	
☒	
☒	
☐	
☐	

ecossistema de
aprendizagem

Checklist de refatoração

O que conferir?

- ☐ Comportamento continuou igual?
- ☐ Arquivos ficaram mais organizados?
- ☐ Nomes melhoraram?
- ☐ Não surgiu dependência nova?
- ☐ O código ficou mais fácil de testar?
- ☐ A explicação da ia bate com o código?



Prática 6: documentação

7

Documentação

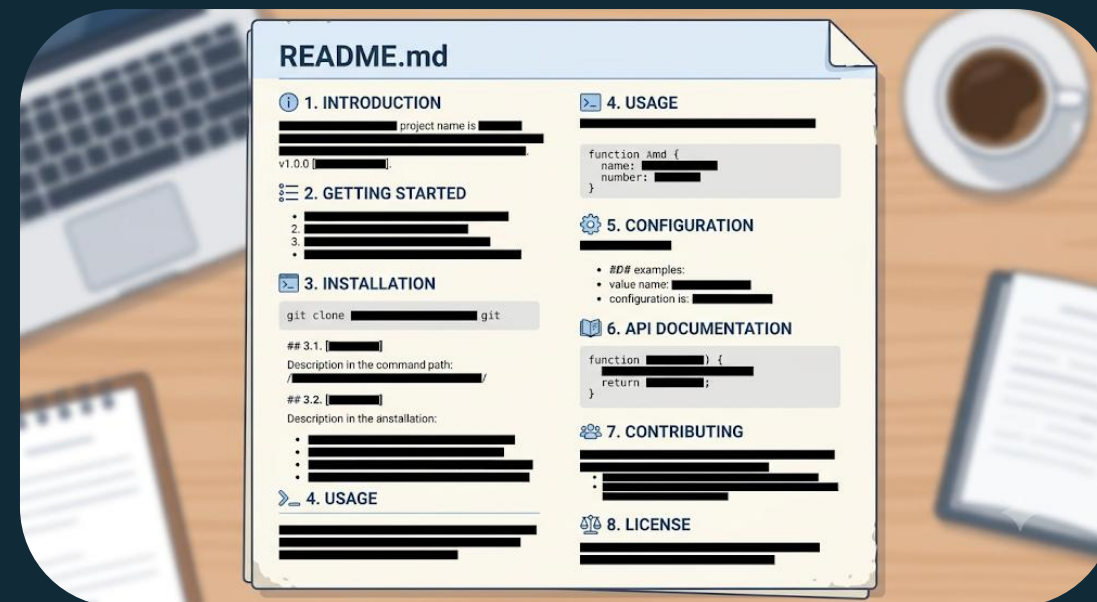


Prompt:

Crie uma documentação curta em Markdown para a API.

Inclua:

- objetivo
- rotas
- exemplo de resposta
- status HTTP
- como testar manualmente
- limitações da versão mockada



Modelo de documentação esperada



API Mockada de Produtos

Objetivo

Fornecer dados fictícios de produtos para testes de front-end.

Rotas

GET /products

Retorna todos os produtos.

GET /products/:id

Retorna um produto pelo ID.

Erros

- 404: produto não encontrado

Limitações

- dados em memória
- sem autenticação
- sem banco de dados



Fluxo completo



O que fizemos: criamos uma rule, geramos função, revisamos código, criamos API mockada, testamos saídas, refatoramos e documentamos.



2

Atividade



Escolha uma missão



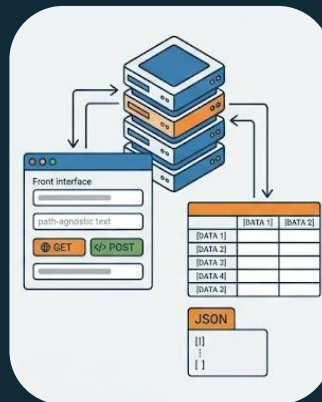
Opção A — Front-end

Criar um componente
ProductCard.



Opção B — Back-end

Criar uma API mockada
de produtos.



Opção C — Automação

Criar uma função que gera
resumo de produtos por
categoria.



Opção A: Front-end



ecossistema de
aprendizagem



Prompt da opção A

Use a rule `natura-product-dev.rules.md`.

Crie um componente `ProductCard` em React com TypeScript.

Regras:

- exibir nome, imagem, categoria e preço
- formatar preço em BRL
- usar fallback para imagem ausente
- desabilitar botão se produto estiver indisponível
- sugerir testes

Opção B: Back-end



ecossistema de
aprendizagem

Prompt da opção B

Use a rule `natura-product-dev.rules.md`.

Crie uma API mockada de produtos com Express.

Rotas:

- GET `/products`
- GET `/products/:id`

Regras:

- produtos fictícios
- status HTTP correto
- tratamento de 404
- código simples
- documentação curta



Opção C: Automação



ecossistema de
aprendizagem



Prompt da opção C

Use a rule `natura-product-dev.rules.md`.

Crie uma função TypeScript chamada `summarizeProductsByCategory`.

Entrada:

lista de produtos.

Saída:

- categoria
- total de produtos
- total disponíveis
- total indisponíveis
- preço médio

Também sugira casos de teste.

Apresentação rápida



- Qual prompt usou?
- O que a ia gerou bem?
- Onde a ia errou?
- Qual ajuste foi necessário?
- O que testaria antes do PR?





Checklist antes do PR

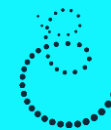
Antes de entregar:

- ☐ Entendi o código?
- ☐ Testei os principais cenários?
- ☐ Removi dados sensíveis?
- ☐ Revisei o diff?
- ☐ Documentei o necessário?
- ☐ Segui os padrões da rule?
- ☐ Consigo explicar para o time?

ecossistema de
aprendizagem

3

Fechamento





Vibe Coding no dia a dia



Use IA para:

- Acelerar tarefas pequenas
- Aprender padrões
- Gerar testes
- Revisar código
- Documentar entregas



Mas nunca pule:

- Entendimento
- Validação
- Segurança
- Revisão



Lembre-se:

Use IA para acelerar o tedioso,
mas nunca delegue sua
responsabilidade técnica.

**Não deixe de
avaliar o nosso
encontro!**

